



NTNU

Institutt for matematiske fag

Løsningsforslag

eksamen i IMAx3011,
IMAx3011 og IMAx3011
Høst 2025

1 La

$$A = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 2 & 0 \\ 1 & 0 & 2 \\ 1 & 0 & 0 \end{bmatrix}$$

a) (2 poeng) Finn en ortogonal basis for kolonnerrommet til A , og vis at basisen du har funnet faktisk er ortogonal.

Løsning: La først

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 2 \\ 2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 2 \\ 0 \\ 2 \\ 0 \end{bmatrix}$$

Vi finner en ortogonal basis $\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\}$ med Gram-Schmidt-prosessen.

La $\mathbf{u}_1 = \mathbf{v}_1$.

Videre er da

$$\mathbf{u}_2 = \mathbf{v}_2 - \frac{\mathbf{u}_1 \cdot \mathbf{v}_2}{\mathbf{u}_1 \cdot \mathbf{u}_1} \mathbf{u}_1 = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}$$

Så finner vi $\mathbf{u}_3$:

$$\mathbf{u}_3 = \mathbf{v}_3 - \frac{\mathbf{u}_1 \cdot \mathbf{v}_3}{\mathbf{u}_1 \cdot \mathbf{u}_1} \mathbf{u}_1 - \frac{\mathbf{u}_2 \cdot \mathbf{v}_3}{\mathbf{u}_2 \cdot \mathbf{u}_2} \mathbf{u}_2 = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}$$

Så sjekker vi at vektorene $\mathbf{u}_1$, $\mathbf{u}_2$ og $\mathbf{u}_3$ faktisk er ortogonale ved å regne ut

$$\mathbf{u}_1 \cdot \mathbf{u}_2 = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot (-1) = 0$$

$$\mathbf{u}_1 \cdot \mathbf{u}_3 = 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 + 1 \cdot (-1) = 0$$

$$\mathbf{u}_2 \cdot \mathbf{u}_3 = 1 \cdot 1 + 1 \cdot (-1) + (-1) \cdot 1 + (-1) \cdot (-1) = 0$$

En ortogonal basis for kolonnerrommet til A er dermed gitt ved

$$\left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \right\}$$

Merk at her er det egentlig uendelig mange gyldige svar. Vi kan gange alle vektorene med ikke-null skalarer siden slik skalering ikke endrer på ortogonalitet.

b) (2 poeng) Finn en matrise Q med ortonormale kolonner og en øvre triangulær matrise R slik at $A = QR$.

Løsning: Vi har allerede funnet en ortogonal basis for kolonnerrommet til A så vi trenger bare

å normalisere disse vektorene:

$$\begin{aligned}\mathbf{q}_1 &= \frac{\mathbf{u}_1}{\|\mathbf{u}_1\|} = \frac{1}{\sqrt{4}} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 1/2 \\ 1/2 \\ 1/2 \end{bmatrix} \\ \mathbf{q}_2 &= \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|} = \frac{1}{\sqrt{4}} \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 1/2 \\ -1/2 \\ -1/2 \end{bmatrix} \\ \mathbf{q}_3 &= \frac{\mathbf{u}_3}{\|\mathbf{u}_3\|} = \frac{1}{\sqrt{4}} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1/2 \\ -1/2 \\ 1/2 \\ -1/2 \end{bmatrix}\end{aligned}$$

Dermed får vi

$$Q = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

Så kan vi beregne R som

$$R = Q^T A = \begin{bmatrix} 2 & 2 & 2 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

En løsning med python kan bruke feks `numpy.linalg.qr`, men må også inneholde sjekk av ortogonalitet enten for hånd eller med kode.

c) (2 poeng) Finn projeksjonen av vektoren $\mathbf{b} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 2 \end{bmatrix}$ på kolonnerommet til A .

Løsning: Projeksjonen av vektoren $\mathbf{b}$ på kolonnerommet til A kan vi beregne som

$$P_{\text{Col}(A)}(\mathbf{b}) = QQ^T \mathbf{b} = \begin{bmatrix} 3/4 \\ 5/4 \\ 5/4 \\ 7/4 \end{bmatrix}$$

Alternativt kan vi beregne projeksjonen på kolonnerommet til A ved å projisere på hver vektor i den ortonormale (eller ortogonale) basisen og summere disse. Vi velger her den ortonormale basisen fordi det gir litt enklere regning siden $\mathbf{q}_i \cdot \mathbf{q}_i = 1$ for $i = 1, 2, 3$.

$$\begin{aligned}P_{\text{Col}(A)}(\mathbf{b}) &= P_{\mathbf{q}_1}(\mathbf{b}) + P_{\mathbf{q}_2}(\mathbf{b}) + P_{\mathbf{q}_3}(\mathbf{b}) \\ &= (\mathbf{q}_1 \cdot \mathbf{b})\mathbf{q}_1 + (\mathbf{q}_2 \cdot \mathbf{b})\mathbf{q}_2 + (\mathbf{q}_3 \cdot \mathbf{b})\mathbf{q}_3 \\ &= \begin{bmatrix} 5/4 \\ 5/4 \\ 5/4 \\ 5/4 \end{bmatrix} + \begin{bmatrix} -1/4 \\ -1/4 \\ 1/4 \\ 1/4 \end{bmatrix} + \begin{bmatrix} -1/4 \\ 1/4 \\ -1/4 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 3/4 \\ 5/4 \\ 5/4 \\ 7/4 \end{bmatrix}\end{aligned}$$

2] Gitt den symmetriske matrisen $A = \begin{bmatrix} a & 1 \\ 1 & 1 \end{bmatrix}$ der $a > 1$.

a) (2 poeng) Vis at matrisen A er en positiv definit matrise.

(Hint: Bruk gjerne følgende resultat: en 2x2 matrise $B = \begin{bmatrix} c & d \\ e & f \end{bmatrix}$ er positiv definit hvis og bare hvis $c > 0$ og $\det(B) = cf - de > 0$.)

Løsning: Bruker hintet. Hvis $a > 1$, så er også $a > 0$. Finner nå determinanten:

$$\det(A) = a \cdot 1 - 1 \cdot 1 = a - 1.$$

Så siden $a > 1$, er også $\det(A) = a - 1 > 0$. Tilsammen får vi da vha hintet at A er positiv definit.

b) (2 poeng) Finn en øvre-triangulær matrise X slik at $X^T X = A$.

Løsning: Her skal vi finne en Cholesky-faktorisering til den symmetriske positiv definite matrisen A . Bruker formel fra formelarket og finner:

$$A = \begin{bmatrix} a & \mathbf{b}^T \\ \mathbf{b} & A_1 \end{bmatrix} = \begin{bmatrix} a & 1 \\ 1 & 1 \end{bmatrix}$$

Så første rad til X er

$$[\sqrt{a} \quad \mathbf{u}] \text{ med } \mathbf{u} = \frac{1}{\sqrt{a}} \mathbf{b} = \frac{1}{\sqrt{a}}.$$

Resten er da $A_1 - \mathbf{u}\mathbf{u}^T = 1 - \frac{1}{\sqrt{a}} \cdot \frac{1}{\sqrt{a}} = 1 - \frac{1}{a}$. Cholesky faktoriseringen til denne resten blir $V = \sqrt{1 - \frac{1}{a}}$.

Tilsammen får vi da:

$$X = \begin{bmatrix} \sqrt{a} & \mathbf{u}^T \\ \mathbf{0} & V \end{bmatrix} = \begin{bmatrix} \sqrt{a} & \frac{1}{\sqrt{a}} \\ 0 & \sqrt{1 - \frac{1}{a}} \end{bmatrix}$$

(Dette er veldefinert, siden $\sqrt{1 - \frac{1}{a}} \in \mathbb{R}$ når $a > 1$.)

- 3 I denne oppgaven skal det brukes jacobimetoden for å tilnærme en løsning til det lineære liknings-systemet $A\mathbf{x} = \mathbf{b}$ med

$$A = \begin{bmatrix} 8 & 2 & 2 \\ 2 & 6 & 1 \\ 2 & 1 & 5 \end{bmatrix} \quad \text{og} \quad \mathbf{b} = \begin{bmatrix} 12 \\ 19 \\ 0 \end{bmatrix}.$$

- a) (2 poeng) Har matrisen A egenskaper som gjør at jacobimetoden konvergerer? Begrunn svaret.

Løsning: Matrisen A er strengt diagonal dominant, siden

$$|8| > |2| + |2|$$

$$|6| > |2| + |1|$$

$$|5| > |2| + |1|.$$

Derfor konvergerer jacobimetoden.

- b) (2 poeng) Fullfør Python-funksjonen `jacobi_solver` ved å erstatte alle deler merka med ???. Funksjonen tar inn:

- en matrise A ,
- en vektor $\mathbf{b}$,
- en startvektor $\mathbf{x}_0$,
- og et antall iterasjoner `n_iter`.

Funksjonen returnerer en tilnærmet løsning $\mathbf{x}_{k+1} = D^{-1}(\mathbf{b} - (L + U)\mathbf{x}_k)$ til det lineære liknings-systemet $A\mathbf{x} = \mathbf{b}$.

Bruk deretter koden din med `n_iter= 5` og $\mathbf{x}_0 = [0, 0, 0]^T$ for å finne en tilnærmet løsning $\mathbf{x}_5$. Oppgi svaret avrundet til 4 desimaler.

Lim inn den ferdige koden din i tekstfeltet i Inspira.

Her kommer koden som skal fullføres og brukes:

Løsning: Finner $\mathbf{x}_5 \approx [1.04041667, 3.04060185, -0.9525]^T$ vha koden nedenfor.

```
1 import numpy as np
2 A = np.array([[8, 2, 2],
3               [2, 6, 1],
4               [2, 1, 5]])
5 b = ???
6 x0 = ???
7
8 def jacobi_solver(A, b, x0, n_iter):
9     # Definer diagonalmatrise D og matrise LU=L+U med A=L+D+U
10    n = x0.size
11    D = np.diag(A)*np.eye(n)
12    Dinv = np.linalg.inv(D)
13    LU = A - D
14
15    # Startverdi for x
16    x = ???
17
18    # Iterasjoner
19    for i in range(???):
20
21        x = ???
22
23    return ???
24
25 # Finn en tilnærming til løsningen til det lineære likningssystemet
26 solution1 = jacobi_solver(???)
27 print("Solution:", solution1)
```

```
1  import numpy as np
2
3  A = np.array([[8, 2, 2],
4                [2, 6, 1],
5                [2, 1, 5]])
6  b = np.array([12,19,0])
7  x0 = np.array([0,0,0])
8
9  def jacobi_solver(A, b, x0, n_iter):
10     # Definer diagonalmatrise D og matrise LU=L+U med A=L+D+U
11     n = x0.size
12     D = np.diag(A)*np.eye(n)
13     Dinv = np.linalg.inv(D)
14     LU = A - D
15
16     # Startverdi for x
17     x = x0
18
19     # Iterasjoner
20     for i in range(n_iter):
21         x = Dinv@(b - LU@ x)
22     return x
23
24
25     # Finn en tilnærming til løsningen til det lineære likningssystemet
26     solution1 = jacobi_solver(A, b, x0, 5)
27     print("Solutions:", solution1)
28     %\end{verbatim}
```

- 4 I denne oppgaven skal det brukes konjugerte gradienter for å tilnærme en løsning til det lineære likningssystemet $A\mathbf{x} = \mathbf{b}$ med

$$A = \begin{bmatrix} 8 & 2 & 2 \\ 2 & 6 & 1 \\ 2 & 1 & 5 \end{bmatrix} \quad \text{og} \quad \mathbf{b} = \begin{bmatrix} 12 \\ 19 \\ 0 \end{bmatrix}.$$

- a) (2 poeng) Har matrisen A egenskaper som gjør at vi kan bruke konjugerte gradienter for å løse det lineære likningssystemet? Begrunn svaret. (*Hint: du kan bruke Python funksjonen `numpy.linalg.eig`.*)

Løsning: Matrisen A er symmetrisk fordi $A^T = A$.

Matrisen A er også positiv definit, fordi den har tre positive egenverdier $\lambda_1 = 10.1925824$, $\lambda_2 = 4.8074176$, $\lambda_3 = 4$. Disse finner vi f.eks. vha `print(np.linalg.eig(A))`.

Derfor er matrisen A symmetrisk positiv definit og vi kan anvende konjugerte gradienter for å løse det lineære likningssystemet.

- b) (2 poeng) Fullfør Python-funksjonen `cg_solver` ved å erstatte alle deler merka med `???`. Funksjonen tar inn:

- en matrise A ,
- en vektor $\mathbf{b}$,
- en startvektor $\mathbf{x}_0$,
- og et antall iterasjoner `n_iter`.

Funksjonen bruker konjugerte gradienter og returnerer en tilnærmet løsning $\mathbf{x}_{k+1}$ til det lineære likningssystemet $A\mathbf{x} = \mathbf{b}$.

Bruk deretter koden din med `n_iter=3` for å finne en tilnærmet løsning $\mathbf{x}_3$. Oppgi svaret avrundet til 4 desimaler.

Lim inn den ferdige koden din i tekstfeltet i Inspira.

Her kommer koden som skal fullføres og brukes:

Løsning: Her kommer den ferdige koden:

Dermed får man løsning $\mathbf{x}_3 = [1, 3, -1]^T$.


```
1 import numpy as np
2
3 A = np.array([[8, 2, 2],
4               [2, 6, 1],
5               [2, 1, 5]])
6 b = np.array([12, 19, 0])
7 x0 = np.array([0, 0, 0])
8
9 def cg_solver(A, b, x0, ???):
10
11     # Startverdi for x, r og d
12     x = x0
13     r = b - A@x0
14     d = r.copy()
15
16     # Iterasjoner
17     for k in range(n_iter):
18         # Beregn produktet A*d
19         Ad = A@d
20
21         #Beregn alpha som prikkprodukt av r og r, delt på
22         # prikkproduktet av d og Ad
23         alpha = np.dot(r,r) / np.dot(d,Ad)
24
25         #Beregn en ny x-verdi
26         x = ???
27
28         #Beregn et nytt residual r_new som tilsvarer r_(k+1)
29         r_new = ???
30
31         # Beregn beta som prikkprodukt av r_new og r_new, delt på
32         # prikkproduktet av r og r
33         beta = ???
34
35         # Beregn en ny søkeretning d
36         d = ???
37
38         # Overta det nye residualet for neste iterasjon
39         r = r_new
40
41     return x
42
43
44 # Finn en tilnærming til løsningen til det lineære likningssystemet
45 solution2 = cg_solver(A, b, x0, 3)
46 print("Solution:", solution2)
```

```
1 import numpy as np
2
3 A = np.array([[8, 2, 2],
4               [2, 6, 1],
5               [2, 1, 5]])
6 b = np.array([12, 19, 0])
7 x0 = np.array([0, 0, 0])
8
9 def cg_solver(A, b, x0, n_iter):
10
11     # Startverdi for x, r og d
12     x = x0
13     r = b - A@x0
14     d = r.copy()
15
16     # Iterasjoner
17     for k in range(n_iter):
18         # Beregn produktet A*d
19         Ad = A@d
20
21         #Beregn alpha som prikkprodukt av r og r, delt på
22         # prikkproduktet av d og Ad
23         alpha = np.dot(r,r) / np.dot(d,Ad)
24
25         #Beregn en ny x-verdi
26         x = x + alpha*d
27
28         #Beregn et nytt residual r_new som tilsvarer r_(k+1)
29         r_new = r - alpha*Ad
30
31         # Beregn beta som prikkprodukt av r_new og r_new, delt på
32         # prikkproduktet av r og r
33         beta = np.dot(r_new,r_new) / np.dot(r,r)
34
35         # Beregn en ny søkeretning d
36         d = r_new + beta * d
37
38         # Overta det nye residualet for neste iterasjon
39         r = r_new
40
41     return x
42
43
44 # Finn en tilnærming til løsningen til det lineære likningssystemet
45 solution2 = cg_solver(A, b, x0, 3)
46 print("Solution:", solution2)
```

- 5 I denne oppgaven skal det sammenlignes ulike metoder for løsning av samme likningssystem $B\mathbf{x} = \mathbf{c}$ med

$$B = \begin{bmatrix} 4 & 1 & 1 \\ 1 & 6 & 1 \\ 1 & 1 & 5 \end{bmatrix} \quad \text{og} \quad \mathbf{c} = \begin{bmatrix} -5 \\ 16 \\ 1 \end{bmatrix}.$$

De to ulike metodene som skal sammenlignes er jacobimetoden og konjugerte gradienter.

- a) (1 poeng) Anta at en av metodene etter k iterasjoner gir en tilnærmet løsning $\mathbf{x}_k$. Finn en annen tilnærmet løsning $\mathbf{x}^*$ til likningssystemet ved å bruke python-funksjonen `numpy.linalg.solve`. Anta nå og fremover at $\mathbf{x}^*$ er den eksakte løsningen. Hvilken type feil er feilen $\|\mathbf{x}^* - \mathbf{x}_k\|_2$? (Alternativt kan du også regne ut $\mathbf{x}^*$ for hånd.)

Løsning: Bruker følgende pythonkode:

```
print(np.linalg.solve(A,b))
```

Finner da $\mathbf{x}^* = [-2, 3, 1.34238415 \cdot 10^{-17}]^T$. Antar man at $\mathbf{x}^*$ er den eksakte løsningen, så er $\|\mathbf{x}^* - \mathbf{x}_k\|$ den (absolutte) foroverfeilen.

- b) (1 poeng) Pythonkode som i oppgave 1 og 2 ble kjørt for likningssystemet $B\mathbf{x} = \mathbf{c}$. Etter tre iterasjoner får vi følgende resultater:

- jacobimetoden gir tilnærming $\mathbf{x}_3^J = [-1.93958333, 3.00833333, 0.025]$,
- konjugerte gradienter gir tilnærming $\mathbf{x}_3^K = [-2.00000000e+00, 3.00000000e+00, -2.77555756e-17]$.

Hvilken av de to tilnærmingene, $\mathbf{x}_3^J$ (fra jacobimetoden) eller $\mathbf{x}_3^K$ (fra konjugerte gradienter) er bedre og hvorfor?

Løsning: Beregning av foroverfeilene

$$\|\mathbf{x}^* - \mathbf{x}_3^K\| = \sqrt{(-2 - (-2))^2 + (3 - 3)^2 + (1.34238415 \cdot 10^{-17} - (-2.77555756 \cdot 10^{-17}))^2} \approx 4.1 \cdot 10^{-17}$$

$$\|\mathbf{x}^* - \mathbf{x}_3^J\| = \sqrt{(-2 - (-1.93958333))^2 + (3 - 3.00833333)^2 + (1.34238415 \cdot 10^{-17} - 0.025)^2} \approx 0.066$$

viser at $\mathbf{x}_3^K$ er en bedre løsning enn $\mathbf{x}_3^J$.

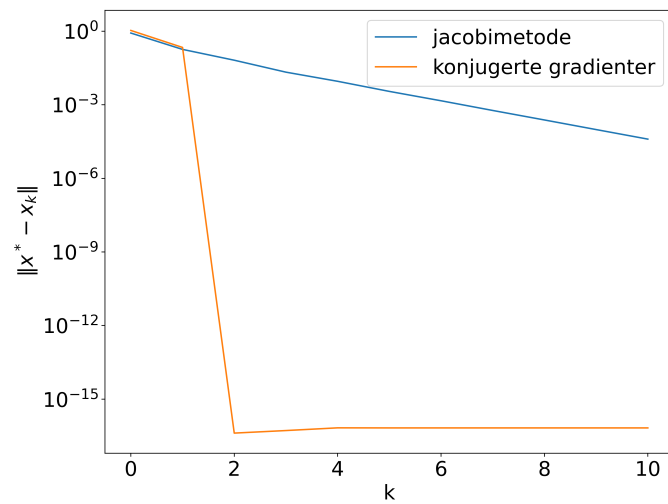
Alternativt kan feilen beregnes vha Python, f.eks.

```
x_star = np.linalg.solve(A,b)
solution1 = np.array([-1.93958333, 3.00833333, 0.025])
feil = np.linalg.norm(solution1-x_star,2)
print(feil)
```

- c) (2 poeng) Følgende figur illustrerer feilen $\|\mathbf{x}^* - \mathbf{x}_k\|$ for $k = 0, 1, \dots, 10$ og for de to numeriske algoritmene:

Hvilken av de to numeriske algoritmene, jacobimetoden eller konjugerte gradienter, er bedre? Begrunn svaret, bl.a. forklar hvordan du tolker ordet *bedre*.

Løsning: For dette likningssystemet er konjugerte gradienter *bedre*, dvs *raskere* enn jacobimetoden. Dette fordi feilen konvergerer raskere mot 0, som man ser i figuren ut i fra stigningstallene. Ca. horisontal linje etter noen iterasjoner svarer til maks presisjon vi kan oppnå ved å bruke double-flyttall.



6 La

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}.$$

a) (1 poeng) Vis at egenverdiene til A er 2 og 5 og finn en tilhørende egenvektor for den dominante egenverdien til A .

Løsning: Vi kan for eksempel finne egenverdier ved å løse $\det(A - \lambda I) = 0$. Da får vi

$$\det(A - \lambda I) = \det \begin{bmatrix} 4 - \lambda & 1 \\ 2 & 3 - \lambda \end{bmatrix} = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = 0$$

Løser vi dette med f.eks. *abc*-formelen, får vi

$$\lambda = \frac{7 \pm \sqrt{7^2 - 4 \cdot 10}}{2} = \frac{7 \pm 3}{2}$$

Med andre ord får vi egenverdiene 5 og 2.

Den dominante egenverdien er den egenverdien som er størst i absoluttverdi, i dette tilfellet 5. Vi finner tilhørende egenvektorer ved å løse ligningen $(A - \lambda I)\mathbf{x} = \mathbf{0}$, der vi setter $\lambda = 5$.

Det blir et ligningssystem med totalmatrise

$$\begin{bmatrix} -1 & 1 & 0 \\ 2 & -2 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Så vi får x_2 som fri variabel og $x_1 = x_2$, og en egenvektor er dermed $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

For ordens skyld kan man sjekke at

$$\begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix} = 5 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

b) (2 poeng) Med startvektor $\mathbf{x}_0 = [1 \ 0]^T$, gjennomfør tre iterasjoner med potensmetoden, der du normaliserer i hvert steg for å finne en tilnærming til en dominant egenvektor. Regner du for hånd, kan du gjerne avrunde til 3 desimaler i hvert steg.

Løsning: La $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$ og la $\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Da får vi først $\mathbf{u}_0 = \mathbf{x}_0$ (fordi $\mathbf{x}_0$ har lengde 1). Så begynner vi å iterere:

$$\begin{aligned} \mathbf{x}_1 &= A\mathbf{u}_0 = \begin{bmatrix} 4 \\ 2 \end{bmatrix}, & \mathbf{u}_1 &= \frac{\mathbf{x}_1}{\|\mathbf{x}_1\|} = \begin{bmatrix} 0.894 \\ 0.447 \end{bmatrix} \\ \mathbf{x}_2 &= A\mathbf{u}_1 = \begin{bmatrix} 4.023 \\ 3.129 \end{bmatrix}, & \mathbf{u}_2 &= \frac{\mathbf{x}_2}{\|\mathbf{x}_2\|} = \begin{bmatrix} 0.789 \\ 0.614 \end{bmatrix} \\ \mathbf{x}_3 &= A\mathbf{u}_2 = \begin{bmatrix} 3.77 \\ 3.42 \end{bmatrix}, & \mathbf{u}_3 &= \frac{\mathbf{x}_3}{\|\mathbf{x}_3\|} = \begin{bmatrix} 0.74 \\ 0.672 \end{bmatrix} \end{aligned}$$

Og vi ser at vi begynner å nærme oss noe som ligger i spenningen til vektoren $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$ som vi fant i oppgave a).

Kommentar: Vha kalkulator og avrunding til 3 desimaler finner man

$$\mathbf{u}_3 = \frac{\mathbf{x}_3}{\|\mathbf{x}_3\|} = \begin{bmatrix} 0.741 \\ 0.672 \end{bmatrix}$$

- c) (1 poeng) Etter tre iterasjoner, finn en tilnærming til dominant egenverdi ved hjelp av Rayleigh-kvotienten. Regner du for hånd, kan du gjerne avrunde til 3 desimaler.

Løsning: Hvis vi lar $\mathbf{x} = \begin{bmatrix} 0.74 \\ 0.672 \end{bmatrix}$ så finner vi en tilnærming til dominant egenverdi slik:

$$\lambda = \frac{\mathbf{x}^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}} \approx 5.042,$$

som er ganske nærme den eksakte egenverdien som er 5.

- d) (2 poeng) La nå $A = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}$. Forklar hvorfor potensmetoden med startvektor $\mathbf{x}_0 = [0 \ 1]^T$ ikke vil konvergere mot en dominant egenvektor.

Løsning: Matrisen har egenverdiene 4 og 3, og altså dominant egenverdi 4. En egenvektor tilhørende egenverdien 4 (altså en dominant egenvektor) vil være $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ mens $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ vil være en egenvektor tilhørende egenverdien 3. Dersom vi bruker sistnevnte som startvektor vil vi etter k iterasjoner ha

$$A^k \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 3^k \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

og dermed vil vi alltid bare bevege oss i retning egenvektoren som ikke er dominant, og aldri konvergere mot den dominante egenvektoren.

- 7 Et lite bakeri har spesialisert seg på smultringer og pepperkaker og forbereder seg til jul. De skal produsere og selge bokser som inneholder enten kun pepperkaker, eller kun smultringer. Produksjonen krever forberedelsestid (laging av deig, kjevling, etc.) og steketid, begge målt i timer. Tabellen under viser hvor mye tid som kreves, samt fortjenesten, per boks:

Type bakst	Forberedelse (timer)	Steketid (timer)	Fortjeneste per boks (kroner)
Pepperkaker	1	1	20
Smultringer	2	1	30

Bakeriet har totalt 8 timer til forberedelser og 5 timer steketid tilgjengelig hver dag. Bakeriet er bare interessert i svar som er heltall - de ønsker ikke å styre med halvfulle bokser.

- a) (2 poeng) Formuler dette som et lineært optimeringsproblem der målet er å maksimere fortjenesten.

Løsning: La x_1 være antall bokser med pepperkaker og la x_2 være antall bokser med smultringer.

Målet er å maksimere fortjenesten, altså å finne

$$\max P = 20x_1 + 30x_2$$

Med opplysningene fra tabellen, samt at vi har 8 timer til forberedelser og 5 timer til steking får vi bibetingelsene

$$\begin{cases} x_1 + 2x_2 \leq 8 & (\text{forberedelse i timer}) \\ x_1 + x_2 \leq 5 & (\text{steketid i timer}) \\ x_1 \geq 0, x_2 \geq 0 & (\text{ikke-negativt antall bokser}) \\ x_1, x_2 \in \mathbb{Z} & (\text{heltall antall bokser}) \end{cases}$$

- b) (2 poeng) Tegn det tillatte området, finn den optimale produksjonsmengden av bokser med pepperkaker og smultringer, og beregn maksimal fortjeneste.

Løsning:

Betingelsene $x_1, x_2 \geq 0$ sier at vi befinner oss i første kvadrant (over x_1 -aksen og til høyre for x_2 -aksen).

Ved å sette hhv. $x_2 = 0$ og $x_1 = 0$ finner vi at linja $x_1 + x_2 = 5$ skjærer x_1 -aksen i $(5, 0)$ og x_2 -aksen i $(0, 5)$.

Tilsvarende finner vi at linja $x_1 + 2x_2 = 8$ skjærer x_1 -aksen i $(8, 0)$ og x_2 -aksen i $(0, 4)$.

Skjæringspunktet mellom de to linjene finner vi ved å sette f.eks.

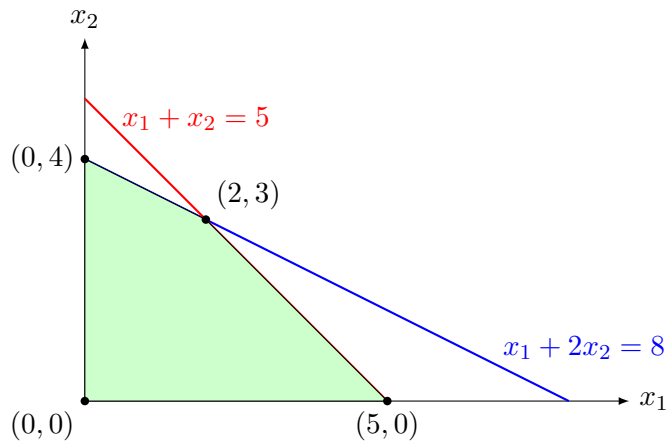
$$x_2 = 5 - x_1 \quad \text{og} \quad x_2 = \frac{8 - x_1}{2}$$

som gir

$$5 - x_1 = \frac{8 - x_1}{2} \implies 10 - 2x_1 = 8 - x_1 \implies x_1 = 2$$

Setter vi dette inn i $x_2 = 5 - x_1$ får vi $x_2 = 3$, og dermed skjæringspunktet $(2, 3)$.

Vi får altså følgende skisse av det tillatte området:



Polygonet har altså fire hjørner vi må sjekke:

$$(0,0) \quad (5,0) \quad (2,3) \quad (0,4)$$

Vi får

$$\begin{aligned} P(0,0) &= 0 \\ P(5,0) &= 100 \\ P(2,3) &= 130 \\ P(0,4) &= 120 \end{aligned}$$

og dermed maksimal fortjeneste hvis vi lager to bokser pepperkaker og tre bokser smultringer. Det gir en fortjeneste på 130 kroner.

Vi legger også merke til at dette hjørnet (i likhet med de andre hjørnene) har heltallskoordinater, så løsningen tilfredsstiller også denne betingelsen.

c) (2 poeng) Formuler dualproblemet til dette lineære optimeringsproblemet. Her skal du ignorere kravet om heltallsløsninger.

Løsning: For å finne dualproblemet noterer vi oss at vi skal maksimere funksjonen $\mathbf{c}^T \mathbf{x}$ med $\mathbf{c} = [20 \quad 30]^T$. Bibetingelsene gir $A\mathbf{x} \leq \mathbf{b}$ med

$$A = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \quad \text{og} \quad \mathbf{b} = \begin{bmatrix} 8 \\ 5 \end{bmatrix}$$

Da vi har to bibetingelser, får vi to dualvariabler, y_1 og y_2 . Siden begge bibetingelser er “mindre enn eller lik”-betingelser, så får vi $y_1 \geq 0$ og $y_2 \geq 0$. Da vi har to variabler i primærproblemet, får vi to bibetingelser i dualproblemet, og siden primærvariablene er positive, blir disse “større enn eller lik”-betingelser. Vi finner dem ved

$$A^T \mathbf{y} \geq \mathbf{c}$$

$$\begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \geq \begin{bmatrix} 20 \\ 30 \end{bmatrix}$$

Som gir betingelsene $y_1 + y_2 \geq 20$ og $2y_1 + y_2 \geq 30$.

Siden primærproblemet er et maksimeringsproblem blir dualproblemet et minimeringsproblem, og funksjonen som skal minimeres er

$$\mathbf{b}^T \mathbf{y} = 8y_1 + 5y_2$$

Oppsummer er altså dualproblemet

$$\min 8y_1 + 5y_2 \quad \text{slik at} \quad \begin{cases} y_1 + y_2 \geq 20 \\ 2y_1 + y_2 \geq 30 \\ y_1, y_2 \geq 0 \end{cases}$$

8] Vi betrakter funksjonen $f: \mathbb{R}^2 \rightarrow \mathbb{R}$ gitt som

$$f(\mathbf{x}) = f(x_1, x_2) = x_1^2 + 2x_2^2 - 4x_1 + 4x_2.$$

a) (2 poeng) Bruk gradientmetoden med steglengde $\alpha = 0.2$ for å tilnærme et minimum til funksjonen $f(\mathbf{x})$. Bruk startverdi $\mathbf{x}_0 = [0, 0]^T$ og utfør tre iterasjoner.

1. Fullfør følgende pythonkode ved å fylle inn spørsmålstegetene ???. Lim deretter inn din ferdige kode i tekstfeltet i Inspira.

```

1  import numpy as np
2
3  x = np.array([0,0])
4  alfa = ???
5
6  # definer målfunksjonen som funksjon av x[0] og x[1]
7  def f(x):
8      return ???
9
10 # 3 iterasjoner av gradientmetoden
11 for i in range(3):
12     sokretning = - np.array([2*x[0]-4, 4*x[1]+4])
13     x = ???
14     print(x, f(x))

```

2. Hvilke tilnærminger finner pythonkoden til minimumspunktet og minimumsverdien?

Løsning: Her er den ferdige koden: Basert på utskriften til Python ser vi at

```

1
2  import numpy as np
3  x = np.array([0,0])
4  alfa = 0.2
5
6  # definer målfunksjonen som funksjon av x[0] og x[1]
7  def f(x):
8      return x[0]**2+2*x[1]**2 -4*x[0]+4*x[1]
9
10 # 3 iterasjoner av gradientmetoden
11 for i in range(3):
12     sokretning = - np.array([2*x[0]-4, 4*x[1]+4])
13     x = x + alfa*sokretning
14     print(x, f(x))

```

$$\mathbf{x}_1 = [0.8, -0.8]^T \text{ med}$$

$$f(\mathbf{x}_1) = -4.48$$

$$\mathbf{x}_2 = [1.28, -0.96]^T \text{ med}$$

$$f(\mathbf{x}_2) = -5.4784$$

$$\mathbf{x}_3 = [1.568, -0.992]^T \text{ med}$$

$$f(\mathbf{x}_3) = -5.813248$$

b) (1 poeng) Konvergerer gradientmetoden for $\mathbf{x}_0 = [0, 0]^T$ og $\alpha = 0.2$? Begrunn svaret kort.

Løsning: Vi ser at avstanden mellom tilnærmingene $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ blir mindre og mindre med hver iterasjon. Derfor konvergerer gradientmetoden for $\mathbf{x}_0 = [0, 0]^T$ og $\alpha = 0.2$.

c) (1 poeng) Hvordan oppfører $\{\mathbf{x}_k\}$ seg når man velger steglengde $\alpha = 0.9$? Begrunn svaret ved bruk av pythonkoden din.

Løsning: Erstatte man `alfa = 0.2` med `alfa = 0.9`, så får man følgende resultat fra Python koden:

$$\begin{array}{ll} \mathbf{x}_1 = [3.6, -3.6]^T \text{ med} & f(\mathbf{x}_1) = 10.08 \\ \mathbf{x}_2 = [0.72, 5.76]^T \text{ med} & f(\mathbf{x}_2) = 87.0336 \\ \mathbf{x}_3 = [3.024, -18.576]^T \text{ med} & f(\mathbf{x}_3) = 612.880128 \end{array}$$

For $\alpha = 0.9$ vokser funksjonsverdiene og $\{x_k\}$ konvergerer ikke, men divergerer.

d) (2 poeng) La $\mathbf{p} = [1, -1]^T$ og $\mathbf{x}_0 = [0, 0]^T$. Finn den perfekte steglengden α_0^* som minimerer

$$\phi(\alpha) = f(\mathbf{x}_0 + \alpha\mathbf{p}).$$

Løsning: Først finner vi det nye punktet

$$\mathbf{x}_0 + \alpha\mathbf{p} = \begin{bmatrix} 0 + \alpha \cdot 1 \\ 0 + \alpha \cdot (-1) \end{bmatrix} = \begin{bmatrix} \alpha \\ -\alpha \end{bmatrix}.$$

Så finner vi

$$\phi(\alpha) = f(\alpha, -\alpha) = \alpha^2 + 2(-\alpha)^2 - 4\alpha + 4(-\alpha) = 3\alpha^2 - 8\alpha.$$

Dette er en funksjon av en variabel, som vi kan minimere ved å løse $\phi'(\alpha) = 0$, altså

$$\phi'(\alpha) = 6\alpha - 8 = 0$$

$$\text{så } \alpha_0^* = \frac{4}{3} > 0 \text{ er den beste steglengden.}$$

- 9 Musen Martha har bestemt seg for å kartlegge hvordan motivasjonen hennes i løpet av dagen påvirkes av hvor mange ostebiter hun ser om morgenen. Hun fører en uke med nøye observasjoner. Høyere tall betyr høyere motivasjon.

På den siste dagen får Martha lite søvn og er i generelt dårlig dagsform. Hun inkluderer likevel denne målingen i datasettet. Hun registrerer følgende verdier:

x (antall ostebiter)	y (motivasjon)
1	2
2	4
3	6
4	7
5	9
100	1

Hun ønsker å lage en lineær modell som beskriver sammenhengen mellom x og y :

$$y = ax + b$$

og bestemmer seg for å bruke minste kvadraters metode.

- a) (2 poeng) Bruk minste kvadraters metode til å finne regresjonslinjen

$$y = ax + b$$

for datasettet over. Sett opp normallikningene og løs for a og b . Det må komme fram tydelig fra besvarelsen din hvordan normallikningene ser ut. Regner du for hånd, kan du gjerne avrunde til 3 desimaler i hvert steg.

- b) (2 poeng) Estimer feilen som du gjør når du approksimerer observasjoner med den lineære modellen $y = ax + b$. Vurder om regresjonsmodellen du fant i forrige deloppgave virker realistisk med tanke på den observerte sammenhengen i datasettet. Drøft hvilke forhold i datasettet som kan føre til at regresjonsmodellen blir misvisende eller ikke representativ.

Løsning:

Vi har datasettet

$$(x_i, y_i) = (1, 2), (2, 4), (3, 6), (4, 7), (5, 9), (100, 1),$$

og ønsker å tilpasse en lineær modell

$$y = ax + b$$

ved minste kvadraters metode.

a) Minste kvadraters metode

Metode 1: Normallikninger

For modellen $y = ax + b$ er normallikningene

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i, \\ a \sum x_i + b n = \sum y_i. \end{cases}$$

Vi beregner summene:

$$\begin{aligned} \sum x_i &= 115, & \sum y_i &= 29, \\ \sum x_i^2 &= 10055, & \sum x_i y_i &= 201, & n &= 6. \end{aligned}$$

Normallikningene blir dermed

$$\begin{cases} 10055a + 115b = 201, \\ 115a + 6b = 29. \end{cases}$$

Vi løser systemet ved eliminasjon og får

$$a \approx -0.045, \quad b \approx 5.696.$$

Regresjonslinjen er derfor

$$y \approx -0.045x + 5.696.$$

Metode 2: Matriseformulering

Vi skriver modellen på matriseform

$$A \begin{pmatrix} a \\ b \end{pmatrix} \approx \mathbf{y},$$

der

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \\ 5 & 1 \\ 100 & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 2 \\ 4 \\ 6 \\ 7 \\ 9 \\ 1 \end{pmatrix}.$$

Normallikningene kan da skrives som

$$A^\top A \begin{pmatrix} a \\ b \end{pmatrix} = A^\top \mathbf{y}.$$

Vi får

$$A^\top A = \begin{pmatrix} 10055 & 115 \\ 115 & 6 \end{pmatrix}, \quad A^\top \mathbf{y} = \begin{pmatrix} 201 \\ 29 \end{pmatrix}.$$

Dette gir samme ligningssystem som over og samme løsning.

b) Estimering av feil og vurdering av modellen

For regresjonslinjen

$$y \approx -0.045x + 5.696$$

beregner vi residualene

$$r_i = y_i - (ax_i + b).$$

Vi får følgende (avrundet til tre desimaler):

x	y	$ax + b$	r_i
1	2	5.651	-3.651
2	4	5.606	-1.606
3	6	5.561	0.439
4	7	5.516	1.484
5	9	5.471	3.529
100	1	1.196	-0.196

Den totale feilen målt ved summen av kvadrerte residualer blir

$$\sum r_i^2 \approx (-3.651)^2 + (-1.606)^2 + (0.439)^2 + (1.484)^2 + (3.529)^2 + (-0.196)^2 \approx 30.7.$$

Dette er en relativt stor feil sammenliknet med variasjonen i observasjonene. (Husker noen hva MSE/RMSE er, betraktes disse også som riktige feilestimer.)

Den lineære regresjonsmodellen som er funnet ved minste kvadraters metode gir en negativ stigning, noe som strider mot den tydelige positive sammenhengen mellom x og y som observeres for de fleste datapunktene. Dette viser at modellen i liten grad reflekterer den underliggende trenden i datasettet.

Årsaken til dette er observasjonen $(100, 1)$, som skiller seg kraftig fra de øvrige både når det gjelder forklaringsvariabel og respons. Denne observasjonen har høy leverage og får dermed uforholdsmessig stor innflytelse på regresjonslinjen.

En mulig tilnærming kunne vært å undersøke uteliggeren nærmere, for eksempel ved å vurdere om den skyldes målefeil, spesielle omstendigheter eller forhold som ikke er representative for normalen. I dette tilfellet er det oppgitt at Martha var i dårlig dagsform, noe som kan gi grunnlag for å behandle observasjonen separat.

Et alternativ er å utelate uteliggeren og tilpasse en ny regresjonsmodell basert på de resterende datapunktene. Dette ville sannsynligvis gi en positiv stigning som bedre samsvarer med den observerte sammenhengen mellom motivasjon og antall ostebiter.

Samlet sett viser dette eksemplet at minste kvadraters metode må brukes med forsiktighet, særlig for små datasett der enkeltobservasjoner kan dominere løsningen.

- 10** Hensikten med denne oppgaven er å vise at du kan lese og forstå spesifikasjonen av et binært flyttallsformat. All nødvendig informasjon står i teksten.

Et binært flyttall t skal skrives som

$$t = \pm m \times 2^e,$$

der mantissa m er et reelt tall mellom 1 og 2 (inkludert 1, men ikke 2), og eksponenten e er et heltall. Halvpresisjonsformatet bruker **16 bit**:

- 1 bit: fortegn
- 5 bit: eksponent (bias = 15)
- 10 bit: mantissa

Tolkning av bitmønstre

Hvis eksponentbits = 0:

$$t = (-1)^{\text{sign}} \times 2^{-14} \times (0.\text{mantissabits})_2$$

Hvis $0 < \text{eksponentbits} < 31$:

$$t = (-1)^{\text{sign}} \times 2^{(\text{eksponentbits})-15} \times (1.\text{mantissabits})_2$$

Hvis eksponentbits = 31:

$$\begin{cases} \text{mantissa} = 0 & \Rightarrow t = \pm\infty \\ \text{mantissa} \neq 0 & \Rightarrow t = \text{NaN} \end{cases}$$

Eksempel

Bitstrengen

$$[1|01101|1100000000]$$

representerer tallet

$$-2^{13-15} \times (1.1100000000)_2 = -\frac{7}{16}.$$

Spørsmål om flyttallsformatet ovenfor

a) (2 poeng)

Definisjon: Maskinepsilon er det minste positive tallet ε slik at

$$1 + \varepsilon$$

gir et anna representerbart tall enn 1 i dette flyttallsformatet.

Bestem maskinepsilon for halvpresisjonsformatet. Bruk bitmønstrene for 1 og det nærmeste tallet over 1 i forklaringa.

b) (2 poeng)

Hvordan lagres tallet

$$N = 12345 = (11000000111001)_2$$

i halvpresisjonsformatet?

Forklar kort:

- hvordan tallet normaliseres
- hvordan eksponenten bestemmes
- hvordan mantissaen (10 bit) konstrueres

- om avrunding er nødvendig (IEEE round-to-nearest, ties-to-even) *IEEE-regelen round-to-nearest, ties-to-even innebærer at tall avrundes til det nærmeste representerbare tallet, og dersom verdien ligger nøyaktig midt mellom to alternativer, avrundes det til det som har en jevn (even) siste bit*

Gi til slutt hele 16-bit-representasjonen.

c) (2 poeng) Vi ønsker å lagre tallet

$$x = 1 + 2^{-12} + 2^{-15}.$$

1. Forklar hvorfor dette tallet ikke kan representeres eksakt i halv-presisjonsformatet.
2. Finn de to nærmeste representerbare tallene rundt x .
3. Forklar hvilke bitmønstre disse tallene har, og avgjør hvilket av dem IEEE-avrunding velger. Du skal bruke den samme avrundingsregelen "IEEE round-to-nearest, ties-to-even".

Løsning:

a) Maskin-epsilon

Maskin-epsilon er definert som det minste tallet $\varepsilon > 0$ slik at

$$1 + \varepsilon \neq 1$$

i halv-presisjonsformatet.

Tallet 1 lagres som:

$$\text{sign} = 0, \quad \text{eksponent} = 01111_2 (= 15), \quad \text{mantissa} = 0000000000.$$

Det nærmeste tallet over 1 har mantissa

$$0000000001_2,$$

altså

$$1 + 2^{-10}.$$

Dermed er maskin-epsilon

$$\varepsilon = 2^{-10}.$$

—

Løsning: Representasjon av heltallet $N = 12345$ i halvpresisjon

1. Binær representasjon

Vi har tallet:

$$N = 12345 = (11000000111001)_2.$$

Dette er et 14-bits binært tall.

2. Normalisering

Vi skriver tallet på formen $1.m \times 2^e$:

$$11000000111001_2 = 1.1000000111001_2 \times 2^{13}.$$

Dermed er eksponenten:

$$e = 13.$$

3. Lagret eksponent

Halvpresisjon bruker bias = 15:

$$E = e + \text{bias} = 13 + 15 = 28.$$

I binær form:

$$28 = (11100)_2.$$

4. Mantissa (10 bit) og korrekt avrunding

Den normaliserte mantissaen er:

$$1.\underbrace{1000000111}_{10 \text{ bit}}001$$

De første 10 bit etter komma beholdes:

$$f = 1000000111$$

Restbitene (de som kastes) er:

$$r = 001_2 = 1/8 = 0.125$$

ULP på engelsk står for Unit in the Last Place, og rundt 1 er det samme som maskinepsilon: En ULP i mantissaområdet nær denne verdien er:

$$\text{ULP} = 2^{-10}.$$

Halvparten av en ULP er:

$$\frac{1}{2}\text{ULP} = 2^{-11}.$$

Vi sammenligner restverdien med halv ULP:

$$0.125 < 0.5.$$

Dermed er vi i første tilfelle av IEEE 754 round-to-nearest:

$$r < \frac{1}{2}\text{ULP} \Rightarrow \text{rund ned.}$$

Ingen økning skjer i mantissaen:

$$\text{mantissa} = 1000000111.$$

5. Endelig representasjon

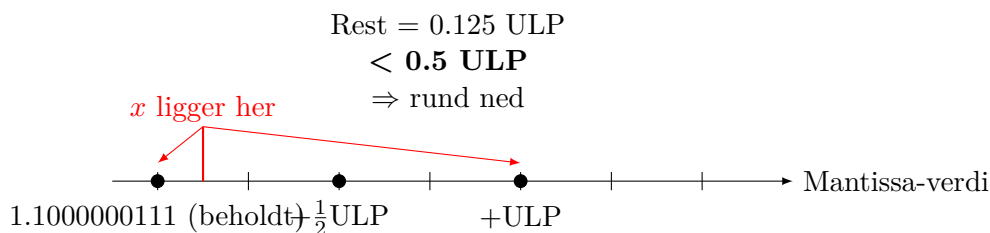
$$\text{sign} = 0$$

$$\text{eksponent} = 11100$$

$$\text{mantissa} = 1000000111$$

Dermed lagres N slik:

$$\boxed{0 \ 11100 \ 1000000111}$$



Figuren viser:

- mantissa etter avkorting (venstre punkt),
- neste representerbare mantissa (til høyre),
- midtpunktet (halv ULP),
- den faktiske verdien før avrunding (rød strek), som ligger klart til venstre for halvpunktet.

Dermed er IEEE 754-avrundingen entydig: **rund ned**.

Løsning til del c)

Vi ønsker å lagre tallet

$$x = 1 + 2^{-12} + 2^{-15}.$$

Dette er et tall svært nær 1, og vi må analysere det i forhold til presisjonen til halv-presisjonsformatet.

1. Hvorfor x ikke kan representeres eksakt

Normaliserte tall rundt 1 har representasjonsform:

$$1.\text{mantissa} \times 2^0.$$

Mantissaen har 10 bit, så minste steg i mantissaen (ULP) er:

$$\text{ULP}(1) = 2^{-10}.$$

Dette betyr at alle representerbare tall i intervallet $[1, 2)$ er av formen:

$$1 + k \cdot 2^{-10}, \quad k \in \mathbb{Z}.$$

Nå betrakter vi selve forskjellen mellom x og 1:

$$x - 1 = 2^{-12} + 2^{-15}.$$

For å sammenligne dette med ULP, skriver vi det i forhold til 2^{-10} :

$$2^{-12} = 2^{-10} \cdot 2^{-2} = 2^{-10} \cdot \frac{1}{4},$$

$$2^{-15} = 2^{-10} \cdot 2^{-5} = 2^{-10} \cdot \frac{1}{32}.$$

Dermed:

$$x - 1 = 2^{-10} \left(\frac{1}{4} + \frac{1}{32} \right) = 2^{-10} \left(\frac{8}{32} + \frac{1}{32} \right) = 2^{-10} \cdot \frac{9}{32}.$$

Sammenlign $\frac{9}{32}$ med 1:

$$\frac{9}{32} = 0.28125.$$

Dermed ligger x mellom to representerbare tall:

$$1 + 0 \cdot 2^{-10} = 1, \quad 1 + 1 \cdot 2^{-10} = 1 + 2^{-10}.$$

Siden $x - 1$ ikke er et helt multiplum av 2^{-10} , er x ikke representerbart.

2. De to nærmeste representerbare tallene

De mulige representerbare tallene rundt 1 er:

$$1, \quad 1 + 2^{-10}, \quad 1 + 2 \cdot 2^{-10}, \dots$$

Vi har vist:

$$x - 1 = 0.28125 \cdot 2^{-10}.$$

Dette gir rangeringen:

$$1 < x < 1 + 2^{-10}.$$

Avstanden til 1 er:

$$d_0 = x - 1 = 0.28125 \cdot 2^{-10}.$$

Avstanden til $1 + 2^{-10}$ er:

$$d_1 = (1 + 2^{-10}) - x = 2^{-10} - (x - 1) = (1 - 0.28125) \cdot 2^{-10} = 0.71875 \cdot 2^{-10}.$$

Dermed:

$$d_0 < d_1.$$

Og det nærmeste tallet er:

$$1.$$

3. Hvilket tall velges ved IEEE-avrunding?

IEEE 754 på halvpresisjon bruker *round to nearest, ties to even*.

Siden:

$$x - 1 = 0.28125 \cdot 2^{-10}, \quad \text{og dette er ikke en tie (0.5 ULP),}$$

og x ligger *klart nærmere* 1 enn $1 + 2^{-10}$,

får vi:

$$\boxed{x \text{ avrundes til } 1}.$$

4. Bitrepresentasjon

Tallet 1 har halvpresisjonsrepresentasjonen:

$$\boxed{0 \ 01111 \ 0000000000}.$$

Dette blir den endelige lagrede verdien.